

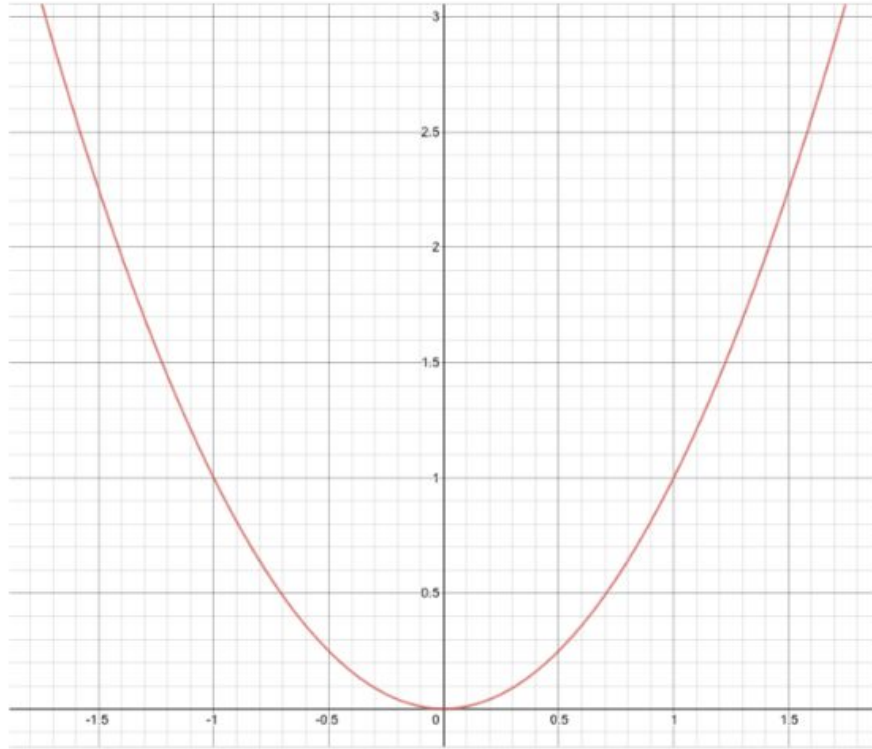
Neural Networks

Seminar - Week 3 - Backward pass, loss functions

Assignment 2 Goals

- Implement loss functions - Mean Square Error, Binary Cross-Entropy
- Implement backward pass and derivatives for
 - Linear layer
 - Activation functions
 - Loss functions
- Implement backward pass for Multi-layer Perceptron

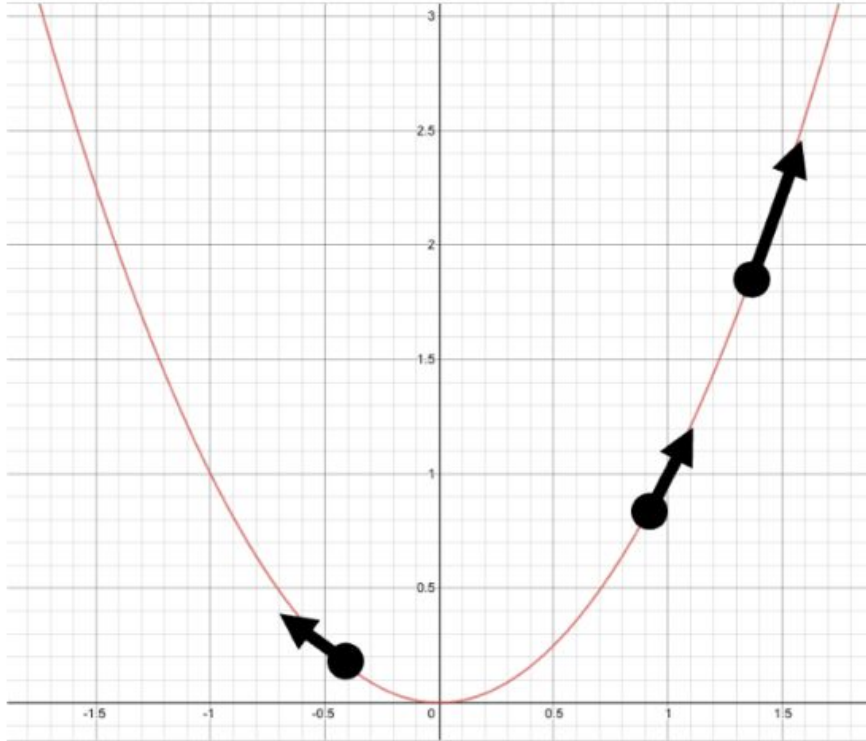
Training of neural network - Gradient Descent



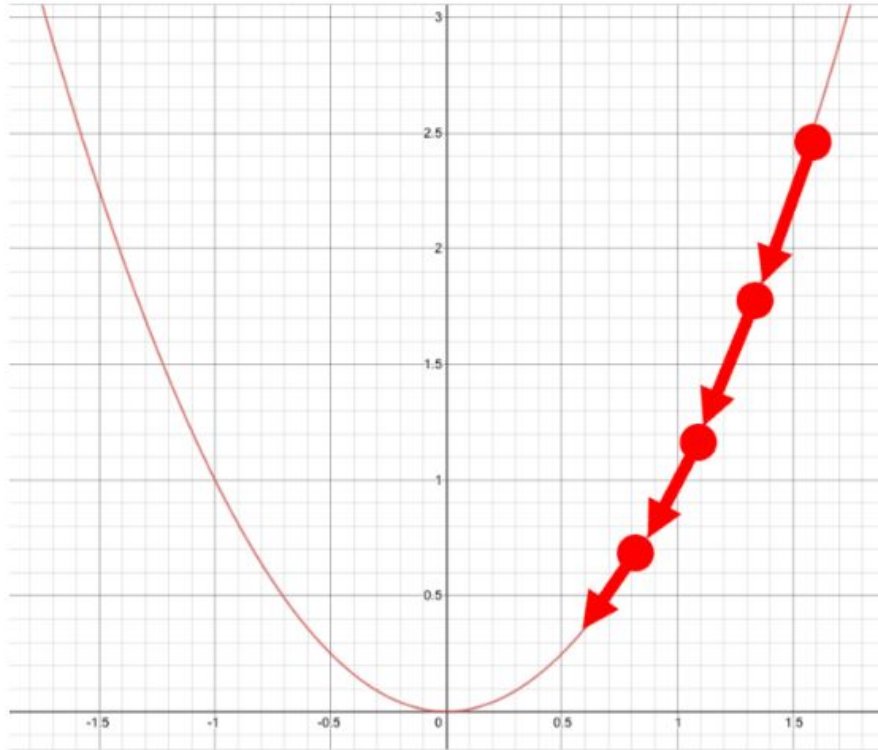
$$p = (x_1, x_2, x_3)$$
$$f(p) = \dots$$

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f(p)}{\partial x_1} \\ \frac{\partial f(p)}{\partial x_2} \\ \frac{\partial f(p)}{\partial x_3} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Training of neural network - Gradient Descent



Training of neural network - Gradient Descent



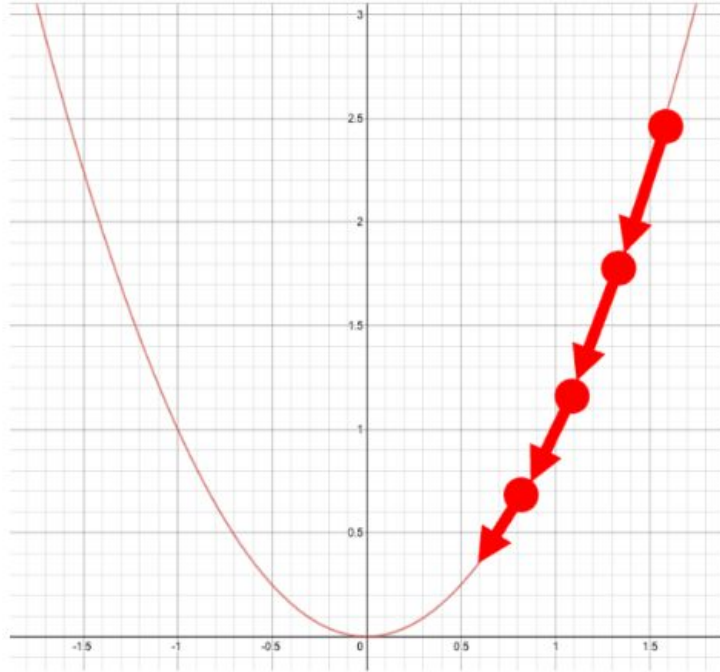
Update rule

$$x := x - \alpha \nabla f(x)$$

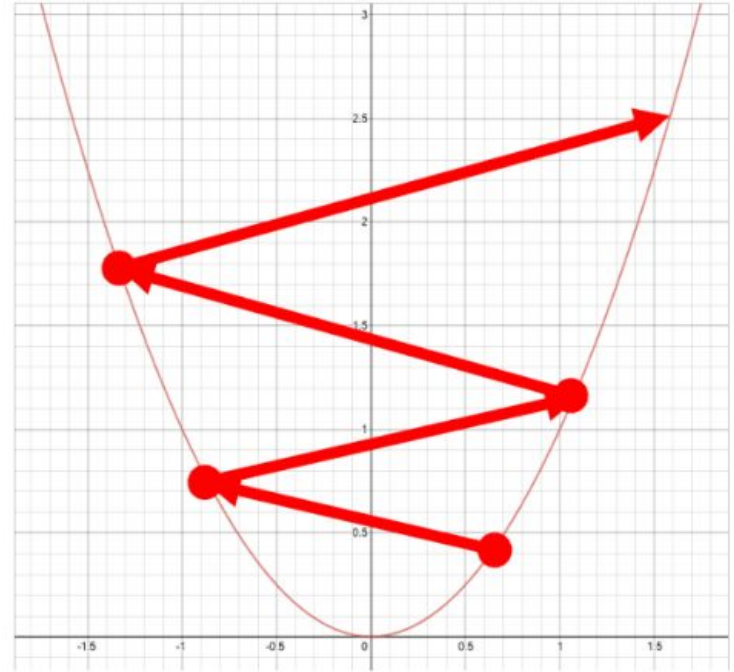
direction of descent - $-\nabla f(x)$

step size, learning rate - α

Training of neural network - Gradient Descent



$\alpha = 0.3$



$\alpha = 1.5$

Loss and Cost

Let:

Training dataset - $\mathcal{X} = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$

Function (model) for estimation of y - $\hat{y} = h(x)$

- **Loss function**

- Measures, how good does the function estimate y for the **given sample**

$$\mathcal{L}(\hat{y}, y) = \mathcal{L}(h(x), y)$$

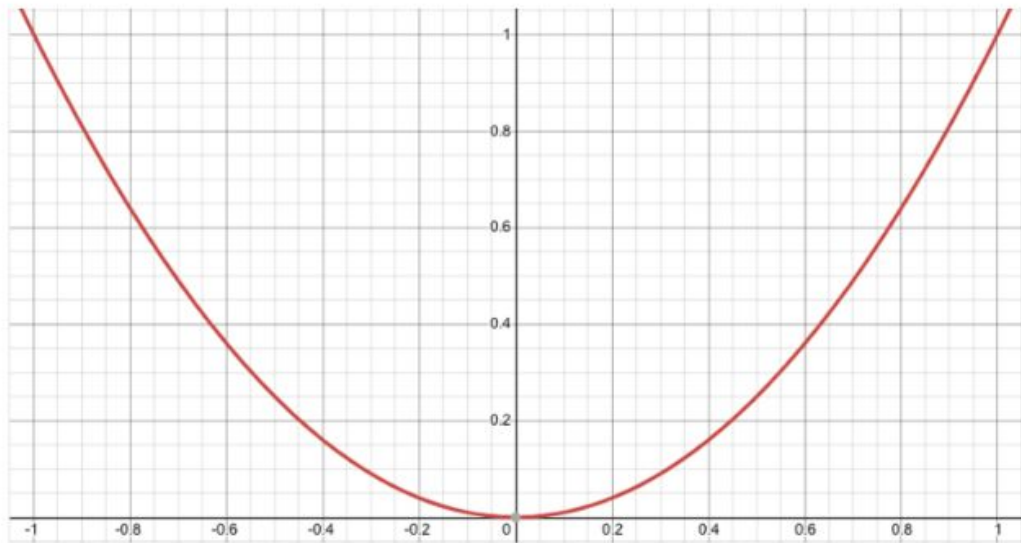
- **Cost function**

- Measures, how good does the function estimate y for the **entire data set**

$$\frac{1}{m} \sum_i^m \mathcal{L}(h(x_i), y_i)$$

Loss Functions

Square Error - L2 Loss



$$\mathcal{L}(\hat{y}, y) = (y - \hat{y})^2$$

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = -2(y - \hat{y})$$

Loss Functions

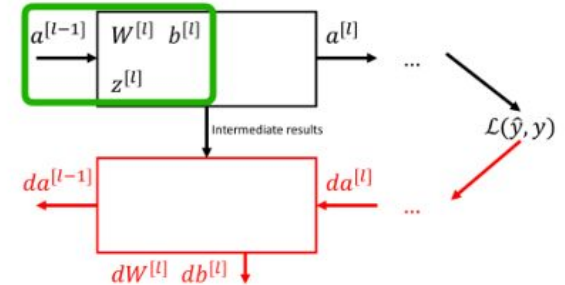
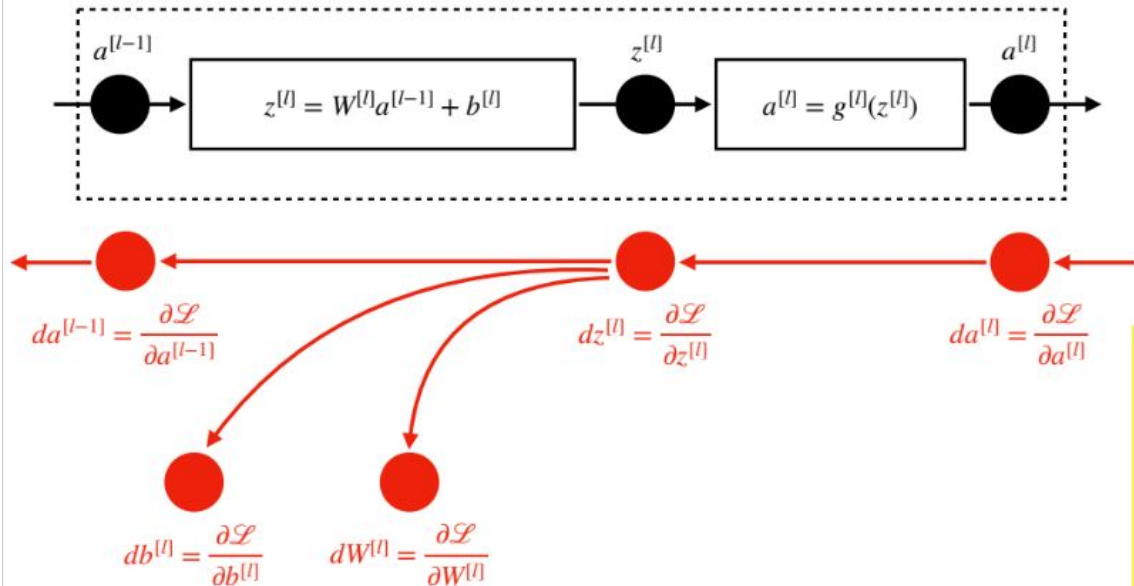
Binary Cross Entropy - BCE Loss

$$\text{BCE}(y, \hat{y}) = - \left[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y}) \right]$$

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}}$$

Projects

Backward pass



$$\begin{aligned} dz^{[l]} &= g^{[l]'}(z^{[l]}) * da^{[l]} \\ da^{[l-1]} &= W^{[l]T} dz^{[l]} \\ db^{[l]} &= dz^{[l]} \\ dW^{[l]} &= dz^{[l]} a^{[l-1]T} \end{aligned}$$

Projects

Backward pass - vectorized

$$dZ^{[2]} = g^{[2]'}(Z^{[2]}) * dA^{[2]}$$

$$dA^{[1]} = W^{[2]T} dZ^{[2]}$$

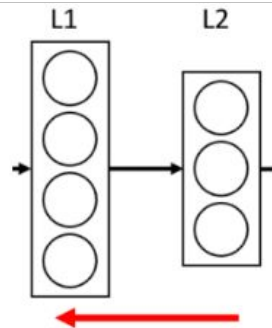
$$db^{[2]} = \text{mean}(dZ^{[2]})$$

m

$$\begin{bmatrix} db^{[2]} \\ (3 \times 1) \end{bmatrix} = \frac{1}{m} \begin{bmatrix} \dots & \dots \\ \dots & dZ^{[2]} & \dots \\ (3 \times m) \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ 1 \\ \vdots \\ (m \times 1) \end{bmatrix}$$

$$\left. \begin{aligned} dZ^{[l]} &= g^{[l]'}(Z^{[l]}) * dA^{[l]} \\ dA^{[l-1]} &= W^{[l]T} dZ^{[l]} \\ db^{[l]} &= \text{mean}(dZ^{[l]}) \\ dW^{[l]} &= \text{mean}(dZ^{[l]} A^{[l-1]T}) \end{aligned} \right\} m$$



Projects

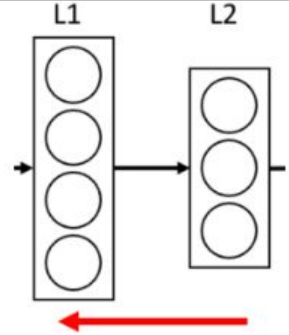
Backward pass - vectorized

$$dZ^{[2]} = g^{[2]'}(Z^{[2]}) * dA^{[2]}$$

$$dA^{[1]} = W^{[2]T} dZ^{[2]}$$

$$db^{[2]} = \text{mean}(dZ^{[2]})$$

$$dW^{[2]} = \text{mean}(dZ^{[2]} A^{[1]T})$$



$$\begin{bmatrix} dW^{[2]} \\ (3 \times 4) \end{bmatrix} = \frac{1}{m} \begin{bmatrix} \dots & \dots \\ \dots & dZ^{[2]} & \dots \\ (3 \times m) \end{bmatrix} \begin{bmatrix} - \\ \dots \\ \dots \\ \dots \end{bmatrix} A^{[1]T} \begin{bmatrix} - \\ \dots \\ \dots \\ \dots \end{bmatrix}$$

$$\begin{aligned} dZ^{[l]} &= g^{[l]'}(Z^{[l]}) * dA^{[l]} \\ dA^{[l-1]} &= W^{[l]T} dZ^{[l]} \\ db^{[l]} &= \text{mean}(dZ^{[l]}) \\ dW^{[l]} &= \text{mean}(dZ^{[l]} A^{[l-1]T}) \end{aligned}$$

Week 4 - TASK

- Implement loss functions - Mean Square Error, Binary Cross-Entropy
- Implement backward pass and derivatives for
 - Linear layer
 - Activation functions
 - Loss functions
- Implement backward pass for Multi-layer Perceptron

https://github.com/vgg-fiit/NSIETE_2026